

Critical Evaluation Report

1. Project Methodologies and Design Principles

To develop the Emergency Resource Allocation console application, I needed a thoughtful and planned strategy. As it was a real-world problem application that prioritised and allocated limited emergency resources, I began by organising the project using a software development methodology. I used a linear model; the process started with planning and system design, then implementation and testing, and concluded with self-assessment and reflection. During the development process, I was able to manage code changes incrementally, set up separate branches for individual features, and revert to earlier versions when needed. This not only improved organisation and safety during development but also laid the base for good collaborative practices.

My main objective from the beginning was to write maintainable, understandable code. Therefore, I applied key software design principles that guided my project:

SOLID principles were essential for my system's structure. Each class had a clear purpose, putting into practice the Single Responsibility Principle, which helped keep the logic modular and easier to test. For instance, the `ResourceManager` and `IncidentManager` classes were specific and did not share responsibilities. Similarly, the Open/Closed Principle was created to design the code in a way that could be extended without the necessity of modification of existing files—something that became particularly handy when introducing different resource types. These practices align with established principles of agile and maintainable software design (Martin, 2003;

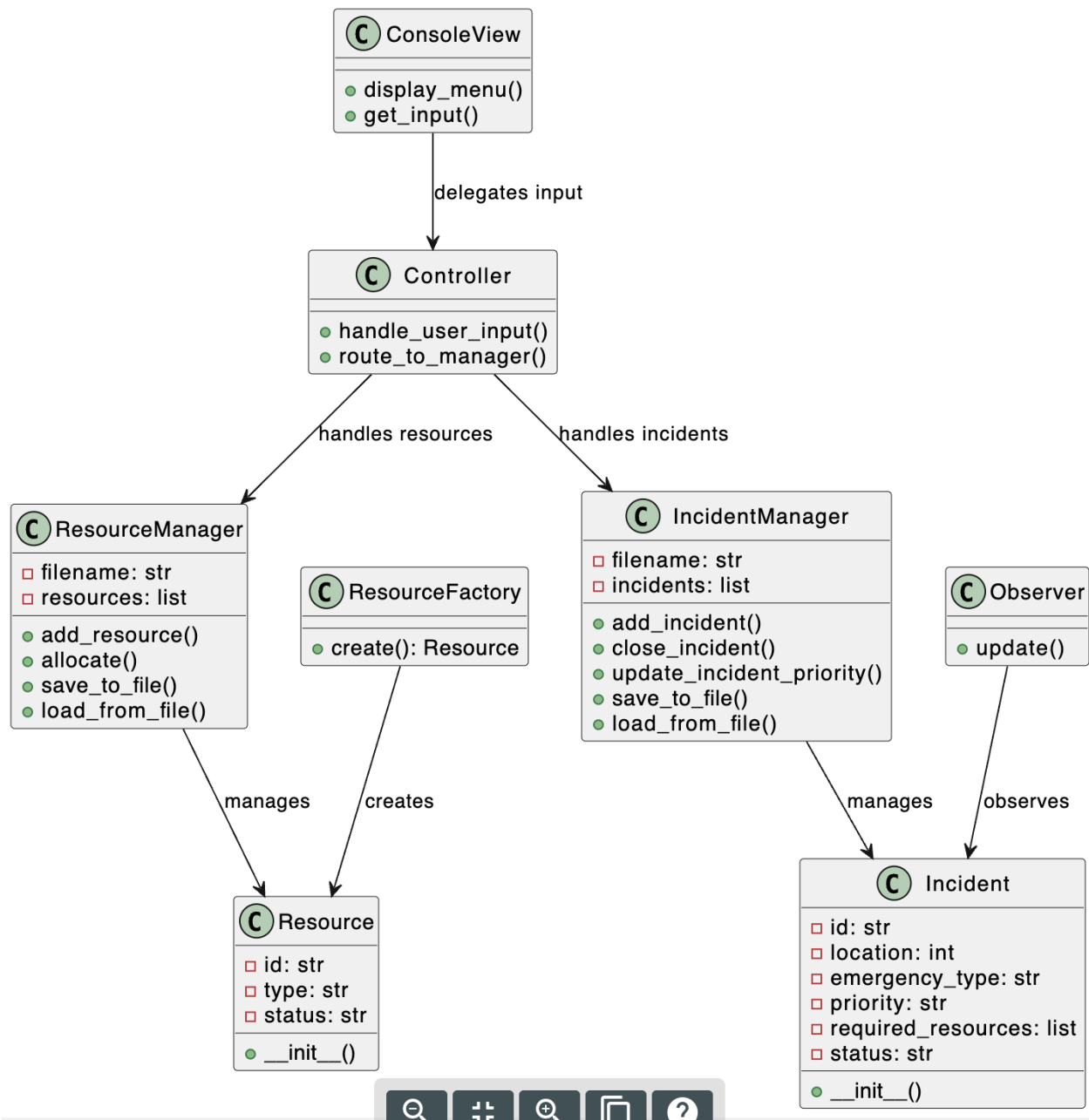
Sommerville, 2016).

The DRY (Don't Repeat Yourself) principle also shaped the codebase. When I realised the patterns emerging, particularly when working with resource updates or incident tracking, I applied the common logic into reusable methods. This reduced redundancy; therefore, it was easier to debug and scale the system.

I also applied **KISS** (Keep It Simple, Stupid), as simple code is easier to read, keep, and explain. It also helps to maintain the software and keep it robust and efficient, mainly for long-term projects or team development.

I designed the application according to a simple Model-View-Controller (MVC) pattern. The data and behaviour of fundamental entities are applied by the models (e.g., Incident, Resource). The controller acts as a link between both, as it coordinates logic and flow, responding to user input, while the view, managed by `console_view.py`, handles all user interactions.

My PlantUML was also based on MVC structure (models, patterns, and controller).



2. Object-Oriented Design and Implementation

I addressed the development of this application with a strong attention to Object-Oriented Programming (OOP). Due to the nature of the project—handling various types of resources, managing multiple incidents, and actively redistributing based on priority—OOP gave a natural and scalable way to structure the code. About the factory design pattern, it helped me create a system for easy resource type instantiation. The code does not require manual ambulance, fire engine, and medical team instantiation because I established a central factory method. The implementation cut down the factory pattern and provided a straightforward method to introduce additional resource types into the system. Applying OOP principles and the Factory Pattern enabled modular extension and code reuse, essential for sustainable development in complex systems (Martin, 2003).

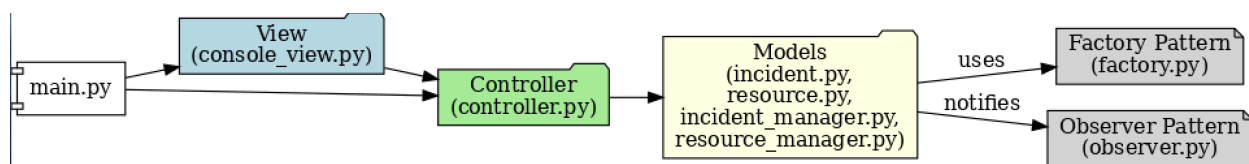
In essence, the application focuses on four key models: Resource, Incident, ResourceManager, and IncidentManager. These classes outline both data and behaviour. For instance, each resource object has attributes like type, location, and availability status, and resources update or allocate themselves to an incident. In the same way, incident objects as requested are given priority, location, and required resources, and interact with the rest of the system through their dedicated methods.

The controller layer functions to direct application behaviour. The controller stays between the view and models to perform tasks like input validation, resource assignment decisions, and system state updates. The Model-View-Controller (MVC) structure directed this separation to maintain both coherence and a clean information

flow. The console view functions to interact with users through input collection and feedback display but avoids direct logic handling, which maintains independent components between layers and, therefore, is easier to maintain. The use of the MVC pattern facilitated clarity and reinforced modularity by standard design architecture recommendations (Sommerville, 2016).

The most interesting technical challenge appeared when creating the resource allocation algorithm. The algorithm bases its incident prioritisation on urgency levels (e.g., 'High', 'Medium', 'Low') and takes into consideration the available resources and their current locations. The system applies competent resource assignment for incident types by giving priority to high-priority incidents and performing reallocations when needed.

Below is the software architecture diagram showing the application's structure:



3. Testing and Quality Assurance

Having a reliable and efficient emergency resource allocation application was a major concern during the development process. Right from the start, I understood that developing a proper and systematic testing plan would not only increase the quality of the code but also provide me with more confidence in making incremental changes.

I mainly concentrated on unit testing, utilising Python's built-in unittest framework. These tests were focused on the most critical components of the system, mainly the resource manager, incident manager, and the resource allocation logic itself. I wrote specific tests for each method to certify that the basic behaviours of the system, including resource allocation, availability tracking, and incident management, worked as I had planned in different scenarios.

Each unit test was written with edge cases in mind. For example, one test reproduced a situation where there were not enough resources for a high-priority incident, and the system dealt well with it by writing an appropriate message instead of failing or crashing.

I performed manual blending checks through the console interface to simulate real-life use cases, although the project depth did not require thorough system testing or integration. The tests verified that controller-model-view interactions were logical and the application behaved correctly in typical workflow scenarios.

The code's modular design, achieved through SOLID principle alignment, provided me the ability to easily identify bugs when they occurred.

4. Future Improvements and Ethical Considerations

While the app meets its goals enough so far, there are some exciting possibilities for future improvements to explore. A key area for enhancement is the user interface at present—it does its job. It is limited to a command-line setup. Adding an interface using a user GUI framework, like Tkinter or PyQt, would improve usability and make the system easier to use, especially for those who aren't tech-savvy.

At a level, the method for putting resources in place is quite straightforward despite being logical and structured around incident priorities. Some possible improvements in the future involve adopting decision-making frameworks. For example, permitting the inclusion of geolocation details could facilitate resource allocation based on proximity. Integrating analytics also helps in predicting demand by analysing incident patterns. These enhancements would make the system more adaptable and responsive to emergency scenarios.

Another important aspect to consider is that the structure of the system, at the moment, cannot handle users at the same time efficiently in high-stress situations when many team members may need to access it simultaneously. To improve this situation and ensure scalability and data persistence for user environments, it would be beneficial to switch to a client-server model or integrate a database-supported framework.

Furthermore, the system's resource reallocation logic may be improved to be more flexible and adaptive than just functional in its present state. Now the system only reallocates resources when new urgent incidents occur; however, future iterations could

take into account factors such as incident frequency, resource fatigue, and the expected probability of future events in particular areas. This adjustment would not only ensure that resource allocation is responsive but also forward-thinking and strategic.

During the development process, I frequently updated the codebase to enhance performance and ease of maintenance. As the application progressed over time, some initial approaches became unnecessary or ineffective.

Ethical Aspects

With the importance of emergency response systems' role in society, it is crucial to uphold values as a core concern. When designing systems that decide and distribute resources, there is always a potential for bias to exist. For instance, if specific types of incidents are consistently given priority based on strict rules or inadequate information, this could lead to the neglect of marginalised communities or areas in need.

To reduce this danger, the existing algorithm was created to be intentionally basic and clear, giving the user the power to make decisions. Nonetheless, upcoming advancements should take into account incorporating protections, like automated audits, escalation procedures, and ongoing feedback mechanisms, to stop abuse and take responsibility. Ethical considerations in algorithm design are fundamental, particularly in autonomous or semi-autonomous systems that impact human lives (Winfield, 2019).

Conclusion

Developing the Emergency Resource Allocation app has been a journey filled with obstacles and moments of self-discovery for me. I drew inspiration from tried and tested techniques to shape this project and incorporated object-oriented design principles with a focus on testing to create a flexible system that serves its purpose effectively. This project has not deepened my understanding of software architecture and design concepts. Also sparked contemplation on the profound influence of technology in critical real-life scenarios.

References

Martin, R.C., 2003. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.

Sommerville, I., 2016. *Software Engineering*. 10th ed. Pearson Education.

Winfield, A.F.T., 2019. Ethics in the design and implementation of autonomous systems. *Proceedings of the IEEE*, 107(3), pp.508–517. doi:10.1109/JPROC.2019.2896018.